

Modern Android Architecture

MVVM · Clean Architecture · Hilt · Retrofit · Room · Coroutines

🏛️ Clean Arch
🎯 MVVM
⚡ Coroutines
🌊 Flow
💉 Hilt
🌐 Retrofit
🗄️ Room
📚 Paging · Coil
🧪 Testing

Book 5 · 15 chapters · real-world project · exercises & cheat sheet

Contents

01 🧰 The Modern Android Stack 3

02 🏛️ Clean Architecture 7

03 ⚡ Coroutines in Depth 10

04 🌊 Kotlin Flow 14

05 🧠 Domain Layer & Use Cases 18

06 🌐 Retrofit & OkHttp 21

07 🗄️ Room in Depth 25

08 🔁 Repository & Offline-First 29

09 💉 Dependency Injection with Hilt 33

10 🎯 MVVM with Compose 38

11 🧭 Navigation + Hilt 42

12 📚 More Popular Libraries 45

13 🧪 Testing the Architecture 49

14 🧱 Modularization 54

15 🚀 Project: GitHub Repo Explorer 57

★ ⚡ Cheat Sheet 64

✓ ✅ Exercise Solutions 65

CHAPTER 1

The Modern Android Stack



What professional apps are built with today

Most new Android apps share a very similar tech stack, recommended by Google and the community. Learn this stack and you can read and work on almost any modern Android codebase.

Need	Library
Language	Kotlin
UI	Jetpack Compose + Material 3
Architecture	MVVM + Clean Architecture (UDF)
Async	Kotlin Coroutines + Flow
Dependency injection	Hilt (built on Dagger)
Networking	Retrofit + OkHttp + kotlinx.serialization
Database	Room
Key/value storage	DataStore
Images	Coil
Navigation	Navigation Compose (type-safe)
Background work	WorkManager
Lists from network	Paging 3
Testing	JUnit, MockK, Turbine, coroutines-test
Code generation	KSP

Version catalog

All versions live in one file, `gradle/libs.versions.toml`, shared by all modules.

gradle/libs.versions.toml

```
[versions]
kotlin = "2.1.0"
ksp = "2.1.0-1.0.29"
hilt = "2.54"
room = "2.6.1"
retrofit = "2.11.0"
```

```
okhttp = "4.12.0"
lifecycle = "2.8.7"
coroutines = "1.9.0"

[libraries]
hilt-android = { module = "com.google.dagger:hilt-
android", version.ref = "hilt" }
hilt-compiler = { module = "com.google.dagger:hilt-
compiler", version.ref = "hilt" }
hilt-navigation-compose = { module =
"androidx.hilt:hilt-navigation-compose", version =
"1.2.0" }
room-runtime = { module = "androidx.room:room-
runtime", version.ref = "room" }
room-ktx = { module = "androidx.room:room-ktx",
version.ref = "room" }
room-compiler = { module = "androidx.room:room-
compiler", version.ref = "room" }
retrofit = { module =
"com.squareup.retrofit2:retrofit", version.ref =
"retrofit" }
retrofit-kotlinx = { module =
"com.squareup.retrofit2:converter-kotlinx-
serialization", version.ref = "retrofit" }
okhttp-logging = { module =
"com.squareup.okhttp3:logging-interceptor",
version.ref = "okhttp" }
kotlinx-serialization = { module =
"org.jetbrains.kotlinx:kotlinx-serialization-json",
version = "1.7.3" }
lifecycle-runtime-compose = { module =
"androidx.lifecycle:lifecycle-runtime-compose",
version.ref = "lifecycle" }
coil-compose = { module = "io.coil-kt.coil3:coil-
compose", version = "3.0.4" }
coil-network = { module = "io.coil-kt.coil3:coil-
network-okhttp", version = "3.0.4" }

[plugins]
ksp = { id = "com.google.devtools.ksp", version.ref =
"ksp" }
hilt = { id = "com.google.dagger.hilt.android",
version.ref = "hilt" }
kotlin-serialization = { id =
"org.jetbrains.kotlin.plugin.serialization",
version.ref = "kotlin" }
```

> ⚠️ **Watch out**
> These are **example versions**. Libraries update often — Android Studio highlights newer ones; use the latest stable releases. The `ksp` version must match your Kotlin version.

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.kotlin.serialization)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

dependencies {
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
    implementation(libs.hilt.navigation.compose)

    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)

    implementation(libs.retrofit)
    implementation(libs.retrofit.kotlinx)
    implementation(libs.okhttp.logging)
    implementation(libs.kotlinx.serialization)
    implementation(libs.coil.compose)
    implementation(libs.coil.network)
}
```

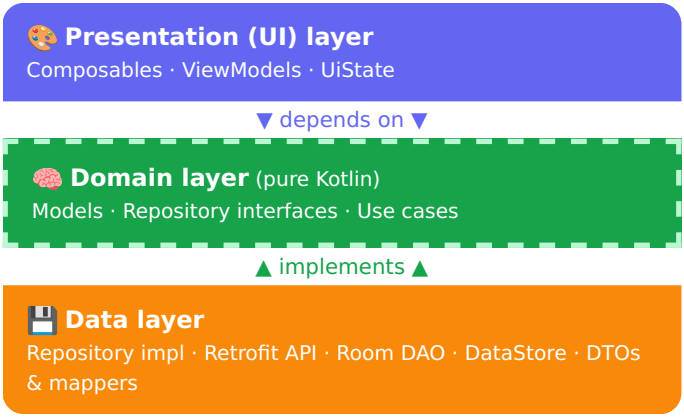
> 📌 **Remember**
> **KSP** (Kotlin Symbol Processing) generates code at compile time for Hilt and Room. It replaced the older, slower `kapt`.

CHAPTER 2

Clean Architecture

Layers, the dependency rule and project structure

Clean Architecture splits an app into layers with clear jobs. The goal: code that is easy to **test**, **change** and **understand**, where business rules don't depend on Android, frameworks or the database.



The dependency rule

Remember

Dependencies point **inward**, toward the domain. The domain knows nothing about Compose, Retrofit or Room. The data layer **implements** interfaces that the domain defines.

Layer	Responsibility	Knows about
Presentation	Show state, handle user events	Compose, ViewModel
Domain	Business rules, app logic	Pure Kotlin only
Data	Get & store data	Retrofit, Room, DataStore

Package structure (feature-first)

Project tree

```
com.example.repos/
 ├─ core/
 │   ├─ network/        ← Retrofit setup, interceptors
 │   ├─ database/       ← AppDatabase
 │   └─ ui/             ← theme, shared composables
 ├─ feature/search/
 │   ├─ data/
 │   │   ├─ remote/     ← GitHubApi, DTOs
 │   │   ├─ local/      ← RepoEntity, RepoDao
 │   │   └─ RepoRepositoryImpl.kt
 │   ├─ domain/
 │   │   ├─ model/Repo.kt
 │   │   ├─ RepoRepository.kt     ← interface
 │   │   └─ SearchReposUseCase.kt
 │   └─ presentation/
 │       ├─ SearchViewModel.kt
 │       ├─ SearchUiState.kt
 │       └─ SearchScreen.kt
 └─ di/                 ← Hilt modules
```

Three kinds of models

Each layer has its own model. **Mappers** convert between them so a change in the API JSON never breaks your UI.

Models.kt

```kotlin
// data/remote — mirrors the JSON
@Serializable
data class RepoDto(
    val id: Long,
    @SerialName("full_name") val fullName: String,
    @SerialName("stargazers_count") val stars: Int
)

// data/local — a database row
@Entity(tableName = "repos")
data class RepoEntity(@PrimaryKey val id: Long,
                      val name: String, val stars:
Int)

// domain — what the app actually needs
data class Repo(val id: Long, val name: String, val
stars: Int)

// mappers (extension functions)
fun RepoDto.toEntity() = RepoEntity(id, fullName,
stars)
fun RepoEntity.toDomain() = Repo(id, name, stars)
```

> 💡 **Tip**
> Small apps can skip the domain layer — Google calls it *optional*. Add it when logic is reused by several ViewModels or gets complex.

CHAPTER 3

Coroutines in Depth

Suspend, scopes, dispatchers and error handling



Coroutines let you write asynchronous code that reads top-to-bottom. They're the backbone of every layer: Retrofit and Room both support `suspend` functions.

suspend functions

Suspend.kt

```
suspend fun loadProfile(id: Int): Profile {
    val user = api.getUser(id)          // waits,
doesn't block
    val posts = api.getPosts(id)        // runs after
user
    return Profile(user, posts)
}
```

📌 Remember

A `suspend` function can pause without blocking the thread. While waiting for the network, the main thread keeps drawing the UI.

Scopes & structured concurrency

Scope	Lifetime
viewModelScope	Cancelled when ViewModel is cleared
lifecycleScope	Cancelled when Activity is destroyed
rememberCoroutineScope()	Cancelled when composable leaves
coroutineScope { }	Waits for all children; one fails → all cancel
supervisorScope { }	Children fail independently

Dispatchers

Dispatchers.kt

```
class ImageRepository @Inject constructor(
    @IoDispatcher private val io: CoroutineDispatcher
) {
    suspend fun save(bitmap: Bitmap, file: File) =
        withContext(io) {                // switch to
background
            file.outputStream().use {

bitmap.compress(Bitmap.CompressFormat.PNG, 100, it)
            }
        }                                // back to
caller's thread
}
```

💡 **Tip**

Main-safety rule: every suspend function should be safe to call from the main thread. The function that does blocking work switches dispatcher itself. Inject dispatchers so tests can replace them.

Parallel work with async

Parallel.kt

```
suspend fun loadDashboard(): Dashboard =
coroutineScope {
    val user = async { api.getUser() }
    val news = async { api.getNews() }
    val weather = async { api.getWeather() }
    Dashboard(user.await(), news.await(),
weather.await())
}   // total time ≈ the slowest call, not the sum
```

Error handling

Errors.kt

```
viewModelScope.launch {
    try {
        val data = repository.load()
        _state.value = UiState.Success(data)
    } catch (e: CancellationException) {
        throw
e                       // never swallow cancellation!
    } catch (e: IOException) {
        _state.value = UiState.Error("Check your
connection")
    } catch (e: HttpException) {
        _state.value = UiState.Error("Server error $
{e.code()}")
    }
}
```

⚠️ **Watch out**

`runCatching { }` and `catch (e: Exception)` also catch `CancellationException`, which can keep cancelled work running. Rethrow it.

Timeouts & cancellation

Cancel.kt

```
val result = withTimeoutOrNull(5_000) {
api.slowCall() }
    ?: return showError("Timed out")

suspend fun processAll(files: List<File>) {
    for (file in files) {
        ensureActive()           // stop early if
cancelled
        process(file)
    }
}

val job = viewModelScope.launch { /* ... */ }
job.cancel()
```

CHAPTER 4

Kotlin Flow



Streams of data: Flow, StateFlow and SharedFlow

A `suspend` function returns **one** value. A **Flow** emits **many values over time**: database updates, search results while typing, a timer.

Type	Behavior	Use for
Flow	Cold: runs only when collected	Room queries, API streams
StateFlow	Hot, always has a current value	UI state in ViewModels
SharedFlow	Hot, no current value, many collectors	Events broadcast
Channel	One-time events, one receiver	Navigation, snackbars

Creating and collecting

FlowBasics.kt

```
fun countdown(from: Int): Flow<Int> = flow {
    for (i in from downTo 0) {
        emit(i)
        delay(1000)
    }
}

viewModelScope.launch {
    countdown(3).collect { println(it) }
}
```

▶ Output

```
3
2
1
0
```

Operators

Operators.kt

```
repository.observeRepos()            //
Flow<List<Repo>>
    .map { list -> list.filter { it.stars > 100 } }
    .map { list -> list.sortedByDescending {
it.stars } }
    .distinctUntilChanged()
    .onEach { Log.d("Repos", "Got ${it.size}") }
    .catch { e -> emit(emptyList()) }   // handle
upstream errors
    .flowOn(Dispatchers.Default)        // upstream
runs here
```

Combining flows

Combine.kt

```
val uiState: StateFlow<HomeUiState> = combine(
    userRepository.user,           // Flow<User>
    settingsRepository.darkMode,   // Flow<Boolean>
    notesRepository.notes          // Flow<List<Note>>
) { user, dark, notes ->
    HomeUiState(user.name, dark, notes)
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = HomeUiState()
)
```

📌 **Remember**

`WhileSubscribed(5_000)` stops upstream work 5 seconds after the UI stops collecting, but survives quick rotations.

Search as you type

Search.kt

```
private val query = MutableStateFlow("")

@OptIn(FlowPreview::class,
ExperimentalCoroutinesApi::class)
val results: StateFlow<List<Repo>> = query
    .debounce(400)                    // wait until
typing stops
    .map { it.trim() }
    .filter { it.length >= 2 }
    .distinctUntilChanged()
    .flatMapLatest { q -> repository.search(q) } //
cancel old search
    .stateIn(viewModelScope,
SharingStarted.WhileSubscribed(5_000),
             emptyList())

fun onQueryChange(q: String) { query.value = q }
```

StateFlow vs SharedFlow

HotFlows.kt

```
private val _state = MutableStateFlow(0)       // needs initial value
val state = _state.asStateFlow()
_state.update { it + 1 }                       // atomic update

private val _events = MutableSharedFlow<String>()
val events = _events.asSharedFlow()
viewModelScope.launch { _events.emit("Saved") }
```

💡 Tip

Collect flows in Compose with `collectAsStateWithLifecycle()`. It stops collecting when the app goes to the background, saving battery.

CHAPTER 5

Domain Layer & Use Cases



Business logic in pure Kotlin

The domain layer is the heart of the app. It contains **models**, **repository interfaces** and **use cases**, with no Android imports at all, so it's fast and easy to unit test.

Domain model

domain/model/Repo.kt

```kotlin
data class Repo(
    val id: Long,
    val name: String,
    val owner: String,
    val description: String?,
    val stars: Int,
    val language: String?,
    val avatarUrl: String
) {
    val isPopular: Boolean get() = stars >= 1_000
}
```

Repository interface

domain/RepoRepository.kt

```kotlin
interface RepoRepository {
    fun observe(query: String): Flow<List<Repo>>
    suspend fun refresh(query: String): Result<Unit>
    suspend fun getById(id: Long): Repo?
}
```

> 📌 **Remember**
> The domain says **what** it needs. The data layer decides **how** (network, cache, both). You can swap implementations without touching the UI.

Use cases

A use case does **one thing**. Using `operator fun invoke` lets you call it like a function.

domain/usecase/UseCases.kt

```
class SearchReposUseCase @Inject constructor(
    private val repository: RepoRepository
) {
    operator fun invoke(query: String):
Flow<List<Repo>> =
        repository.observe(query.trim().lowercase())
}

class GetTrendingReposUseCase @Inject constructor(
    private val repository: RepoRepository
) {
    operator fun invoke(): Flow<List<Repo>> =
        repository.observe("stars:>10000")
            .map { list ->
                list.filter { it.language != null }
                    .sortedByDescending { it.stars }
                    .take(20)
            }
}

// in a ViewModel:
val repos = searchRepos("kotlin")   // looks like a
function call
```

Combining repositories

GetFeedUseCase.kt

```
class GetFeedUseCase @Inject constructor(
    private val posts: PostRepository,
    private val users: UserRepository,
    private val blocked: SettingsRepository
) {
    operator fun invoke(): Flow<List<FeedItem>> =
combine(
        posts.observeAll(), users.observeAll(),
blocked.blockedIds
    ) { postList, userList, blockedIds ->
        val byId = userList.associateBy { it.id }
        postList
            .filter { it.authorId !in blockedIds }
            .mapNotNull { p -> byId[p.authorId]?.let
{ FeedItem(p, it) } }
    }
}
```

> 💡 **Tip**
>
> Name use cases **verb + noun + UseCase**: `GetUserUseCase`, `LogoutUseCase`, `ToggleFavoriteUseCase`.

CHAPTER 6

Retrofit & OkHttp

Talking to REST APIs



Retrofit turns an HTTP API into a Kotlin interface. **OkHttp** is the engine underneath (connections, caching, interceptors). **kotlinx.serialization** converts JSON.

DTOs

data/remote/dto/Dtos.kt

```
@Serializable
data class SearchResponseDto(
    @SerialName("total_count") val totalCount: Int,
    val items: List<RepoDto>
)

@Serializable
data class RepoDto(
    val id: Long,
    val name: String,
    val description: String? = null,
    @SerialName("stargazers_count") val stars: Int,
    val language: String? = null,
    val owner: OwnerDto
)

@Serializable
data class OwnerDto(
    val login: String,
    @SerialName("avatar_url") val avatarUrl: String
)
```

The API interface

data/remote/GitHubApi.kt

```
interface GitHubApi {
    @GET("search/repositories")
    suspend fun search(
        @Query("q") query: String,
        @Query("sort") sort: String = "stars",
        @Query("per_page") perPage: Int = 30
    ): SearchResponseDto

    @GET("repos/{owner}/{name}")
    suspend fun getRepo(
        @Path("owner") owner: String,
        @Path("name") name: String
    ): RepoDto

    @POST("user/repos")
    suspend fun create(@Body body: CreateRepoDto):
RepoDto

    @GET("user")
    suspend fun me(@Header("Authorization") token:
String): Response<UserDto>
}
```

Annotation	Meaning
@GET @POST @PUT @PATCH @DELETE	HTTP method + path
@Path("id")	Fills {id} in the path
@Query("q")	Adds ?q=... to the URL
@Body	Sends an object as JSON
@Header / @Headers	Request headers
Response<T>	Access status code & headers

Building Retrofit with interceptors

Network.kt

```
class AuthInterceptor @Inject constructor(
    private val tokenStore: TokenStore
) : Interceptor {
    override fun intercept(chain: Interceptor.Chain):
Response {
        val token = tokenStore.currentToken()
        val request =
chain.request().newBuilder().apply {
            header("Accept", "application/json")
            if (token != null)
header("Authorization", "Bearer $token")
        }.build()
        return chain.proceed(request)
    }
}

val client = OkHttpClient.Builder()
    .addInterceptor(authInterceptor)
    .addInterceptor(HttpLoggingInterceptor().apply {
        level = if (BuildConfig.DEBUG)
HttpLoggingInterceptor.Level.BODY
                else HttpLoggingInterceptor.Level.NONE
    })
    .connectTimeout(15, TimeUnit.SECONDS)
    .build()

val json = Json { ignoreUnknownKeys = true }

val api: GitHubApi = Retrofit.Builder()
    .baseUrl("https://api.github.com/")       // must
end with /
    .client(client)
    .addConverterFactory(json.asConverterFactory("applic
json".toMediaType()))
    .build()
    .create(GitHubApi::class.java)
```

▶ Logcat (OkHttp)

```
--> GET https://api.github.com/search/repositories?
q=kotlin&sort=stars&per_page=30
<-- 200 OK (412ms)
```

Safe API calls

SafeApiCall.kt

```kotlin
suspend fun <T> safeApiCall(block: suspend () -> T):
Result<T> =
    try {
        Result.success(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: HttpException) {
        Result.failure(ApiError.Http(e.code()))
    } catch (e: IOException) {
        Result.failure(ApiError.NoInternet)
    } catch (e: SerializationException) {
        Result.failure(ApiError.BadData)
    }

sealed class ApiError : Exception() {
    data class Http(val code: Int) : ApiError()
    data object NoInternet : ApiError()
    data object BadData : ApiError()
}
```

⚠️ **Watch out**

Never put secret API keys directly in code; they can be extracted from the APK. Keep secrets on your server, or at least in `local.properties` + `BuildConfig` for low-risk keys.

💡 **Tip**

Alternative: **Ktor Client** by JetBrains. Popular for Kotlin Multiplatform projects sharing networking with iOS.

CHAPTER 7

Room in Depth



Entities, DAOs, relations, converters and migrations

Entity & DAO

data/local/RepoDao.kt

```
@Entity(
    tableName = "repos",
    primaryKeys = ["id", "searchQuery"],
    indices = [Index("searchQuery")]
)
data class RepoEntity(
    val id: Long,
    val searchQuery: String,
    val name: String,
    val owner: String,
    val description: String?,
    val stars: Int,
    val language: String?,
    val avatarUrl: String,
    @ColumnInfo(defaultValue = "0") val isFavorite:
Boolean = false
)

@Dao
interface RepoDao {

@Query("SELECT * FROM repos WHERE searchQuery = :q
ORDER BY stars DESC")
    fun observe(q: String): Flow<List<RepoEntity>>

    @Query("SELECT * FROM repos WHERE id = :id LIMIT
1")
    suspend fun getById(id: Long): RepoEntity?

    @Upsert
    suspend fun upsertAll(items: List<RepoEntity>)

    @Query("DELETE FROM repos WHERE searchQuery = :q")
    suspend fun deleteForQuery(q: String)
```

```
    @Transaction
    suspend fun replaceForQuery(q: String, items:
List<RepoEntity>) {
        deleteForQuery(q)
        upsertAll(items)
    }
}
```

> 📌 **Remember**
> `@Upsert` inserts new rows and updates existing ones. `@Transaction` runs several operations atomically: all or nothing.

Relations (one-to-many)

Relations.kt

```
@Entity data class Author(@PrimaryKey val id: Long,
val name: String)

@Entity(foreignKeys = [ForeignKey(
    entity = Author::class, parentColumns = ["id"],
    childColumns = ["authorId"], onDelete =
ForeignKey.CASCADE)],
    indices = [Index("authorId")])
data class Book(@PrimaryKey val id: Long, val
authorId: Long,
                val title: String)

data class AuthorWithBooks(
    @Embedded val author: Author,
    @Relation(parentColumn = "id", entityColumn =
"authorId")
    val books: List<Book>
)

@Dao interface LibraryDao {
    @Transaction
    @Query("SELECT * FROM Author")
    fun authorsWithBooks():
Flow<List<AuthorWithBooks>>
}
```

Type converters

AppDatabase.kt

```
class Converters {
    @TypeConverter
    fun fromInstant(value: Instant?): Long? =
value?.toEpochMilli()

    @TypeConverter
    fun toInstant(value: Long?): Instant? =
value?.let(Instant::ofEpochMilli)
}

@Database(entities = [RepoEntity::class,
Author::class, Book::class],
          version = 2)
@TypeConverters(Converters::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun repoDao(): RepoDao
    abstract fun libraryDao(): LibraryDao
}
```

Migrations

When you change a table, increase `version` and tell Room how to upgrade existing users' data:

Migrations.kt

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "ALTER TABLE repos ADD COLUMN isFavorite
INTEGER NOT NULL DEFAULT 0")
    }
}

Room.databaseBuilder(context,
AppDatabase::class.java, "app.db")
    .addMigrations(MIGRATION_1_2)
    .build()
```

⚠️ **Watch out**

`fallbackToDestructiveMigration()` deletes all user data on schema change. OK during development, dangerous in production.

💡 **Tip**

Simple changes can use **auto-migrations**: `@Database(version = 2, autoMigrations = [AutoMigration(from = 1, to = 2)])` with schema export enabled.

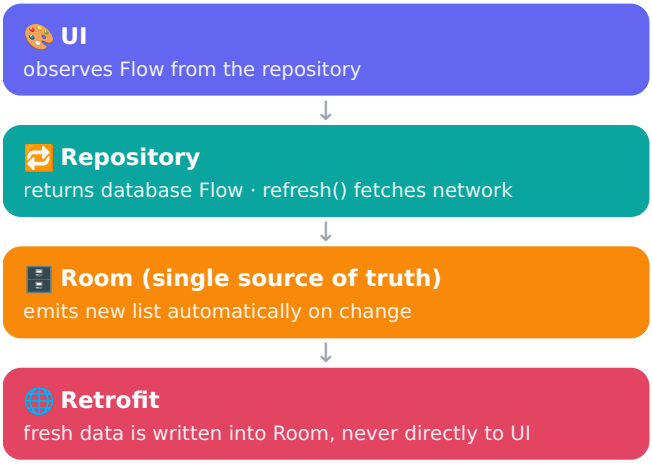
CHAPTER 8

Repository & Offline-First

One source of truth for your data



The **repository** hides where data comes from. In an **offline-first** app, the UI **always reads from the database**; the network only **refreshes** the database.



Implementation

data/RepoRepositoryImpl.kt

```
class RepoRepositoryImpl @Inject constructor(
    private val api: GitHubApi,
    private val dao: RepoDao,
    @IoDispatcher private val io: CoroutineDispatcher
) : RepoRepository {

    override fun observe(query: String):
Flow<List<Repo>> =
        dao.observe(query).map { rows -> rows.map {
it.toDomain() } }

    override suspend fun refresh(query: String):
Result<Unit> =
        withContext(io) {
            safeApiCall { api.search(query) }.map {
response ->
                dao.replaceForQuery(query,
                    response.items.map {
it.toEntity(query) })
            }
        }

    override suspend fun getById(id: Long): Repo? =
        dao.getById(id)?.toDomain()
}
```

Mappers

data/Mappers.kt

```kotlin
fun RepoDto.toEntity(query: String) = RepoEntity(
    id = id, searchQuery = query, name = name,
    owner = owner.login, description = description,
    stars = stars, language = language,
    avatarUrl = owner.avatarUrl
)

fun RepoEntity.toDomain() = Repo(
    id = id, name = name, owner = owner,
    description = description, stars = stars,
    language = language, avatarUrl = avatarUrl
)
```

Why offline-first?

	Benefit
✈️	App works without internet
⚡	Instant start: shows cached data first
🔄	UI updates automatically when data changes
🧪	One source of truth = fewer bugs

Alternative: a Resource wrapper

Resource.kt

```
sealed interface Resource<out T> {
    data object Loading : Resource<Nothing>
    data class Success<T>(val data: T) : Resource<T>
    data class Error(val message: String) :
Resource<Nothing>
}

fun getUser(id: Int): Flow<Resource<User>> = flow {
    emit(Resource.Loading)
    emit(try {
Resource.Success(api.getUser(id).toDomain()) }
         catch (e: IOException) { Resource.Error("No
internet") })
}
```

> 📌 **Remember**
> Use `Resource` for simple network-only screens; use offline-first for content users come back to.

CHAPTER 9

Dependency Injection with Hilt

Dagger made easy for Android



Dependency injection (DI) means a class **receives** what it needs instead of creating it. This makes code testable and loosely coupled.

WhyDI.kt

```
// ❌ Without DI: hard-wired, impossible to test with a fake
class ProfileViewModel : ViewModel() {
    private val repo = UserRepository(Retrofit.Builder()/*...*/)
}

// ✅ With DI: dependencies come from outside
class ProfileViewModel(private val repo: UserRepository) : ViewModel()
```

Dagger is a powerful compile-time DI library. **Hilt** is built on Dagger and removes most of its boilerplate for Android.

1. Setup

App.kt

```
@HiltAndroidApp
class ReposApp : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity() { /* setContent { ... } */ }
```

2. Constructor injection

If Hilt can build all constructor parameters, just add `@Inject`:

Inject.kt

```
class SearchReposUseCase @Inject constructor(
    private val repository: RepoRepository
) { /* ... */ }
```

3. Modules for things you don't own

di/Modules.kt

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @Provides @Singleton
    fun provideJson(): Json = Json {
ignoreUnknownKeys = true }

    @Provides @Singleton
    fun provideOkHttp(): OkHttpClient =
OkHttpClient.Builder()
        .addInterceptor(HttpLoggingInterceptor()
            .apply { level =
HttpLoggingInterceptor.Level.BASIC })
        .build()

    @Provides @Singleton
    fun provideApi(client: OkHttpClient, json: Json):
GitHubApi =
        Retrofit.Builder()
            .baseUrl("https://api.github.com/")
            .client(client)
            .addConverterFactory(
                json.asConverterFactory("application/
json".toMediaType()))
            .build()
            .create(GitHubApi::class.java)
}

@Module
```

```
@InstallIn(SingletonComponent::class)
object DatabaseModule {
    @Provides @Singleton
    fun provideDb(@ApplicationContext ctx: Context):
AppDatabase =
        Room.databaseBuilder(ctx,
AppDatabase::class.java, "repos.db")
            .build()

    @Provides
    fun provideRepoDao(db: AppDatabase): RepoDao =
db.repoDao()
}
```

4. @Binds: interface → implementation

di/RepositoryModule.kt

```
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
    @Binds @Singleton
    abstract fun bindRepoRepository(
        impl: RepoRepositoryImpl
    ): RepoRepository
}
```

5. Qualifiers: two things of the same type

di/Qualifiers.kt

```
@Qualifier @Retention(AnnotationRetention.BINARY)
annotation class IoDispatcher

@Module
@InstallIn(SingletonComponent::class)
object DispatchersModule {
    @Provides @IoDispatcher
    fun provideIo(): CoroutineDispatcher =
Dispatchers.IO
}

class Repo @Inject constructor(
    @IoDispatcher private val io: CoroutineDispatcher)
```

6. ViewModels

Hilt ViewModel

```
@HiltViewModel
class SearchViewModel @Inject constructor(
    private val searchRepos: SearchReposUseCase,
    private val savedState: SavedStateHandle    //
provided automatically
) : ViewModel()

@Composable
fun SearchRoute(vm: SearchViewModel =
hiltViewModel()) { /* ... */ }
```

Components & scopes

Component / scope	Lives as long as
SingletonComponent `@Singleton`	Whole app
ActivityRetainedComponent `@ActivityRetainedScoped`	Survives rotation
ViewModelComponent `@ViewModelScoped`	One ViewModel
ActivityComponent `@ActivityScoped`	One Activity

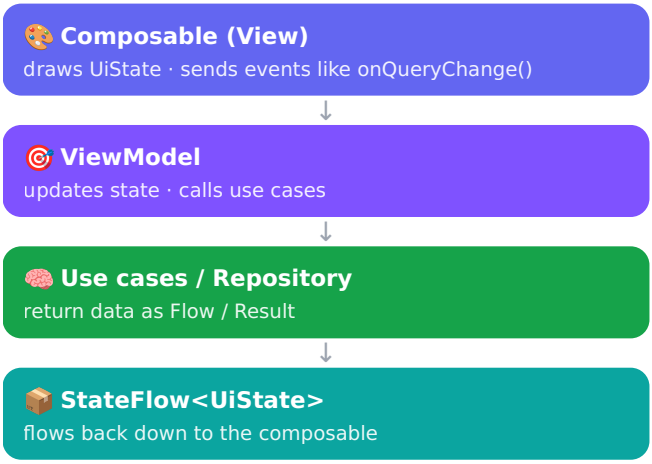
> 💡 **Tip**
> Hilt checks the whole dependency graph at **compile time**. A missing binding is a build error, not a crash in production. **Koin** is a popular lighter alternative that resolves at runtime.

CHAPTER 10

MVVM with Compose

ViewModel, UiState and unidirectional data flow

MVVM = **Model** (data + domain), **View** (composables), **ViewModel** (holds state, handles events). Combined with Compose, it follows **Unidirectional Data Flow**:



1. Model the screen state

SearchUiState.kt

```
data class SearchUiState(
    val query: String = "",
    val repos: List<Repo> = emptyList(),
    val isLoading: Boolean = false,
    val errorMessage: String? = null
) {
    val isEmpty get() = repos.isEmpty() && !isLoading
}
```

> 📌 **Remember**
> One immutable data class = the entire screen. Easy to preview, easy to test, impossible to get into half-updated states.

2. The ViewModel

ProfileViewModel.kt

```
@HiltViewModel
class ProfileViewModel @Inject constructor(
    private val getUser: GetUserUseCase,
    private val logout: LogoutUseCase
) : ViewModel() {

    private val _state =
MutableStateFlow(ProfileUiState())
    val state: StateFlow<ProfileUiState> =
_state.asStateFlow()

    private val _events = Channel<ProfileEvent>()
    val events = _events.receiveAsFlow()      // one-
time events

    init { load() }

    fun load() = viewModelScope.launch {
        _state.update { it.copy(isLoading = true,
error = null) }
        getUser()
            .onSuccess { user ->
                _state.update { it.copy(isLoading =
false, user = user) } }
            .onFailure { e ->
                _state.update { it.copy(isLoading =
false,
                                        error =
e.message) } }
    }

    fun onLogoutClick() = viewModelScope.launch {
        logout()
        _events.send(ProfileEvent.NavigateToLogin)
    }
```

```
}

sealed interface ProfileEvent { data object
NavigateToLogin : ProfileEvent }
```

3. Route + Screen split

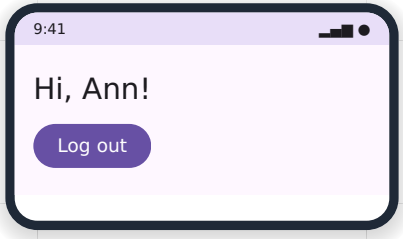
ProfileScreen.kt

```
// Route: connects the ViewModel (not previewable)
@Composable
fun ProfileRoute(onLoggedOut: () -> Unit,
                 vm: ProfileViewModel =
hiltViewModel()) {
    val state by
vm.state.collectAsStateWithLifecycle()

    LaunchedEffect(Unit) {
        vm.events.collect { event ->
            when (event) {
                ProfileEvent.NavigateToLogin ->
onLoggedOut()
            }
        }
    }
    ProfileScreen(state, onRetry = vm::load,
                  onLogout = vm::onLogoutClick)
}

// Screen: pure UI, easy to preview & test
@Composable
fun ProfileScreen(state: ProfileUiState,
                  onRetry: () -> Unit, onLogout: () -
> Unit) {
    when {
        state.isLoading -> CircularProgressIndicator()
        state.error != null -> ErrorView(state.error,
onRetry)
        state.user != null -> Column {
            Text("Hi, ${state.user.name}!")
            Button(onClick = onLogout) { Text("Log
out") }
        }
    }
}
```

```
@Preview @Composable
fun ProfilePreview() = ProfileScreen(
    ProfileUiState(user = User("Ann")), {}, {})
```



📱 What you see

MVVM vs MVI

Pattern	Events	Notes
MVVM	Several public functions (onQueryChange, retry)	Most common, simple
MVI	One function: onAction(Action) with a sealed class	Very explicit, popular in big teams

Mvi.kt

```
// MVI style
sealed interface SearchAction {
    data class QueryChanged(val q: String) :
SearchAction
    data object Retry : SearchAction
}
fun onAction(action: SearchAction) = when (action) {
    is SearchAction.QueryChanged -> query.value =
action.q
    SearchAction.Retry -> refresh()
}
```

CHAPTER 11

Navigation + Hilt

Screens, arguments and scoped ViewModels



Type-safe routes

Routes.kt

```
@Serializable data object SearchRoute
@Serializable data class DetailRoute(val repoId: Long)
@Serializable data object SettingsRoute
```

NavHost

AppNavHost.kt

```
@Composable
fun AppNavHost(navController: NavHostController =
rememberNavController()) {
    NavHost(navController, startDestination =
SearchRoute) {

        composable<SearchRoute> {
            SearchRoute(onRepoClick = { id ->

navController.navigate(DetailRoute(id))
            })
        }

        composable<DetailRoute> {
            // hiltViewModel() gets a ViewModel scoped
to THIS screen
            DetailRoute(onBack =
navController::popBackStack)
        }

        composable<SettingsRoute> { SettingsRoute() }
    }
}
```

Read arguments in the ViewModel

DetailViewModel.kt

```
@HiltViewModel
class DetailViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val repository: RepoRepository
) : ViewModel() {

    private val repoId =
savedStateHandle.toRoute<DetailRoute>().repoId

    val repo: StateFlow<Repo?> = flow {
emit(repository.getById(repoId)) }
        .stateIn(viewModelScope,
SharingStarted.WhileSubscribed(5_000), null)
}
```

📌 **Remember**

Pass only **IDs** between screens, not whole objects. The next screen loads what it needs from the repository, its single source of truth.

Nested graphs & shared ViewModels

NestedGraph.kt

```
@Serializable data object CheckoutGraph
@Serializable data object CartRoute
@Serializable data object PaymentRoute

navigation<CheckoutGraph>(startDestination =
CartRoute) {
    composable<CartRoute> { entry ->
        val parent = remember(entry) {

navController.getBackStackEntry<CheckoutGraph>() }
        val vm: CheckoutViewModel =
hiltViewModel(parent)  // shared
        CartScreen(vm)
    }
    composable<PaymentRoute> { entry ->
        val parent = remember(entry) {

navController.getBackStackEntry<CheckoutGraph>() }

PaymentScreen(hiltViewModel<CheckoutViewModel>(parent))
    }
}
```

💡 Tip

Keep navigation logic out of screens: screens expose callbacks like `onRepoClick`, and only the NavHost knows about `navController`.

CHAPTER 12

More Popular Libraries

Coil, Paging 3, WorkManager, DataStore and friends



Coil — image loading

Coil.kt

```
AsyncImage(
    model = ImageRequest.Builder(LocalContext.current)
        .data(repo.avatarUrl)
        .crossfade(true)
        .build(),
    contentDescription = repo.owner,
    placeholder =
painterResource(R.drawable.placeholder),
    error = painterResource(R.drawable.broken_image),
    contentScale = ContentScale.Crop,
    modifier = Modifier.size(48.dp).clip(CircleShape)
)
```

Paging 3 — endless lists

Paging.kt

```
class RepoPagingSource(
    private val api: GitHubApi, private val query:
String
) : PagingSource<Int, RepoDto>() {

    override suspend fun load(params:
LoadParams<Int>): LoadResult<Int, RepoDto> {
        val page = params.key ?: 1
        return try {
            val items = api.searchPage(query, page,
params.loadSize).items
            LoadResult.Page(
                data = items,
                prevKey = if (page == 1) null else
```

```
page - 1,
                nextKey = if (items.isEmpty()) null
else page + 1)
        } catch (e: IOException) {
            LoadResult.Error(e)
        }
    }

    override fun getRefreshKey(state:
PagingState<Int, RepoDto>) =
        state.anchorPosition?.let {
state.closestPageToPosition(it)?.prevKey?.plus(1) }
}

// ViewModel
val repos = Pager(PagingConfig(pageSize = 30)) {
    RepoPagingSource(api, "kotlin")
}.flow.cachedIn(viewModelScope)

// Compose
val items = vm.repos.collectAsLazyPagingItems()
LazyColumn {
    items(items.itemCount, key = items.itemKey {
it.id }) { i ->
        items[i]?.let { RepoRow(it) }
    }
    if (items.loadState.append is LoadState.Loading) {
        item { CircularProgressIndicator() }
    }
}
```

WorkManager with Hilt

SyncWorker.kt

```
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted context: Context,
    @Assisted params: WorkerParameters,
    private val repository: RepoRepository     //
injected!
) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result =
        if (repository.refresh("kotlin").isSuccess)
Result.success()
        else Result.retry()
}

@HiltAndroidApp
class ReposApp : Application(),
Configuration.Provider {
    @Inject lateinit var workerFactory:
HiltWorkerFactory
    override val workManagerConfiguration:
Configuration
        get() = Configuration.Builder()
            .setWorkerFactory(workerFactory).build()
}
```

📌 Remember

Also add `androidx.hilt:hilt-work` and remove the default WorkManager initializer from the manifest (see the Hilt docs).

DataStore in a repository

SettingsRepository.kt

```
class SettingsRepository @Inject constructor(
    private val dataStore: DataStore<Preferences>
) {
    private val darkKey =
booleanPreferencesKey("dark")
    val darkMode: Flow<Boolean> = dataStore.data.map
{ it[darkKey] ?: false }
    suspend fun setDark(on: Boolean) { dataStore.edit
{ it[darkKey] = on } }
}
```

Other famous names

Library	For
Timber	Better logging (tags automatic, off in release)
kotlinx-datetime	Multiplatform dates & times
Firebase	Auth, Firestore, Crashlytics, Push (FCM)
LeakCanary	Finds memory leaks in debug builds
Ktor Client	Multiplatform networking
Koin	Lightweight DI alternative
Detekt / ktlint	Static analysis & code style
Compose Multiplatform	Share UI with iOS & desktop

CHAPTER 13

Testing the Architecture

Unit tests for ViewModels, repositories and DAOs



Clean layers make testing easy: replace dependencies with **fakes** or **mocks**, then check behavior.

Tool	Purpose
JUnit 4/5	Test runner & assertions
kotlinx-coroutines-test	runTest, test dispatchers
Turbine	Test Flows step by step
MockK	Mocking for Kotlin
Truth / AssertK	Readable assertions
Room in-memory	Real DB for DAO tests

Main dispatcher rule

test/MainDispatcherRule.kt

```
class MainDispatcherRule(
    val dispatcher: TestDispatcher =
UnconfinedTestDispatcher()
) : TestWatcher() {
    override fun starting(description: Description) =
        Dispatchers.setMain(dispatcher)
    override fun finished(description: Description) =
        Dispatchers.resetMain()
}
```

A fake repository

test/FakeRepoRepository.kt

```
class FakeRepoRepository : RepoRepository {
    val data =
MutableStateFlow<List<Repo>>(emptyList())
    var shouldFail = false

    override fun observe(query: String) = data
    override suspend fun refresh(query: String):
Result<Unit> =
        if (shouldFail)
Result.failure(IOException("offline"))
        else Result.success(Unit)
    override suspend fun getById(id: Long) =
        data.value.firstOrNull { it.id == id }
}
```

📌 **Remember**

Prefer **fakes** (small working implementations) over mocks for repositories. Tests become more readable and less fragile.

Testing a use case

test/GetTrendingReposUseCaseTest.kt

```
class GetTrendingReposUseCaseTest {
    @Test
    fun `filters repos without language and sorts by
stars`() = runTest {
        val fake = FakeRepoRepository().apply {
            data.value = listOf(
                repo(id = 1, stars = 10, language =
"Kotlin"),
                repo(id = 2, stars = 99, language =
null),
                repo(id = 3, stars = 50, language =
"Java"))
        }
        val result = GetTrendingReposUseCase(fake)
().first()
        assertEquals(listOf(3L, 1L), result.map {
it.id })
    }
}
```

```
▶ Test results
✔ filters repos without language and sorts by stars
```

Testing a Flow with Turbine

test/SearchViewModelTest.kt

```
@Test
fun `error is shown when refresh fails`() = runTest {
    val repo = FakeRepoRepository().apply {
shouldFail = true }
    val vm = SearchViewModel(SearchReposUseCase(repo),

RefreshReposUseCase(repo),
                            SavedStateHandle())
    vm.uiState.test {
        vm.onQueryChange("kotlin")
        advanceTimeBy(500)                          //
pass debounce
        val state = expectMostRecentItem()
        assertEquals("offline", state.errorMessage)
        cancelAndIgnoreRemainingEvents()
    }
}
```

MockK basics

MockK.kt

```
val api = mockk<GitHubApi>()
coEvery { api.search("kotlin", any(), any()) } returns
    SearchResponseDto(1, listOf(sampleDto))

repository.refresh("kotlin")

coVerify(exactly = 1) { api.search("kotlin", any(),
any()) }
```

DAO test with in-memory Room

androidTest/RepoDaoTest.kt

```
@RunWith(AndroidJUnit4::class)
class RepoDaoTest {
    private lateinit var db: AppDatabase

    @Before fun setup() {
        db = Room.inMemoryDatabaseBuilder(

ApplicationProvider.getApplicationContext(),
            AppDatabase::class.java).build()
    }
    @After fun close() = db.close()

    @Test fun replaceForQuery_removesOldRows() =
runTest {
        val dao = db.repoDao()
        dao.upsertAll(listOf(entity(1, "kotlin"),
entity(2, "kotlin")))
        dao.replaceForQuery("kotlin",
listOf(entity(3, "kotlin")))
        assertEquals(listOf(3L),
dao.observe("kotlin").first().map { it.id })
    }
}
```

CHAPTER 14

Modularization

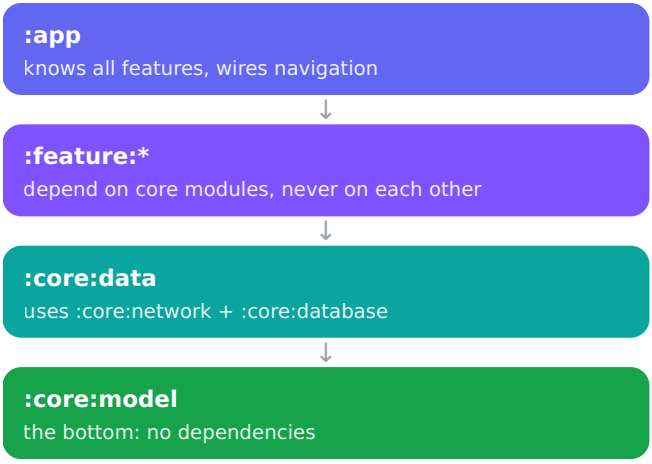
Scaling the codebase for teams



As apps grow, one giant `app` module builds slowly and gets tangled. Big apps split code into **Gradle modules**.

settings.gradle.kts modules

```
:app                    ← MainActivity, NavHost,
Application
:core:model             ← domain models (pure Kotlin)
:core:network           ← Retrofit, OkHttp setup
:core:database          ← Room database, DAOs
:core:data              ← repositories
:core:designsystem      ← theme, shared components
:core:testing           ← fakes, test rules
:feature:search         ← search screen + ViewModel
:feature:detail         ← detail screen + ViewModel
:feature:settings
```



Benefits

	Benefit
⚡	Faster builds: only changed modules recompile
🔒	`internal` hides implementation details
👥	Teams own separate features
♻️	Reuse core modules in other apps

A feature module

feature/search/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

android { namespace = "com.example.feature.search" }

dependencies {
    implementation(project(":core:data"))
    implementation(project(":core:designsystem"))
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
    testImplementation(project(":core:testing"))
}
```

Feature navigation entry

SearchNavigation.kt

```
// in :feature:search
@Serializable data object SearchRoute

fun NavGraphBuilder.searchScreen(onRepoClick: (Long) -
> Unit) {
    composable<SearchRoute> {
SearchRoute(onRepoClick) }
}

// in :app
NavHost(navController, startDestination =
SearchRoute) {
    searchScreen(onRepoClick = {
navController.navigate(DetailRoute(it)) })
    detailScreen(onBack = navController::popBackStack)
}
```

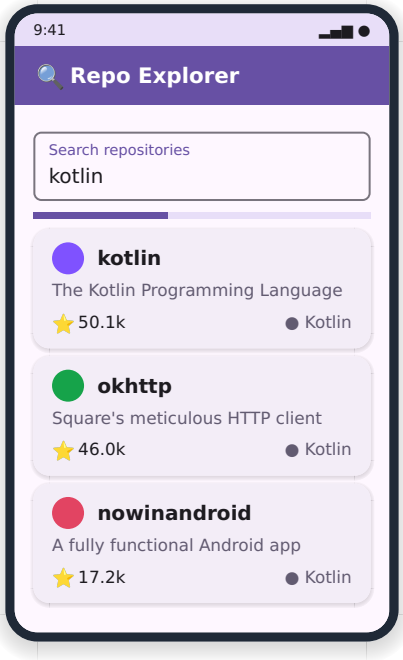
> 💡 **Tip**
> To avoid repeating Gradle config in every module, big projects use **convention plugins** in a `build-logic` folder. Google's open-source **Now in Android** app is the best real example of all of this.

CHAPTER 15

Project: GitHub Repo Explorer

Clean Architecture + MVVM + Hilt + Retrofit + Room + Flow

A complete offline-first app that searches GitHub repositories (public API, no key needed). It uses every concept from this book.



What you see

Files

```
app/src/main/java/com/example/repos/
 ├─ ReposApp.kt · MainActivity.kt
 ├─ di/            NetworkModule · DatabaseModule ·
RepositoryModule
 ├─ data/
 │   ├─ remote/    GitHubApi · Dtos
 │   ├─ local/     RepoEntity · RepoDao · AppDatabase
 │   ├─ Mappers.kt · SafeApiCall.kt
 │   └─ RepoRepositoryImpl.kt
 ├─ domain/        Repo · RepoRepository ·
SearchRepos/RefreshRepos use cases
 └─ presentation/  SearchUiState · SearchViewModel ·
SearchScreen
```

Reuse from earlier chapters: **Dtos** & **GitHubApi** (Ch. 6), **RepoEntity** & **RepoDao** (Ch. 7), **RepoRepositoryImpl** & **mappers** (Ch. 8), **Hilt modules** (Ch. 9), **safeApiCall** (Ch. 6). Below are the remaining files.

Domain

domain/UseCases.kt

```kotlin
class SearchReposUseCase @Inject constructor(
    private val repository: RepoRepository
) {
    operator fun invoke(query: String) =
repository.observe(query.normalize())
}

class RefreshReposUseCase @Inject constructor(
    private val repository: RepoRepository
) {
    suspend operator fun invoke(query: String) =
        repository.refresh(query.normalize())
}

private fun String.normalize() = trim().lowercase()
```

Database & app

AppDatabase.kt / ReposApp.kt / MainActivity.kt

```
@Database(entities = [RepoEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun repoDao(): RepoDao
}

@HiltAndroidApp
class ReposApp : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { MaterialTheme { SearchRoute() } }
    }
}
```

> 📌 **Remember**
>
> Remember `<uses-permission android:name="android.permission.INTERNET"/>` and `android:name=".ReposApp"` in the manifest.

ViewModel

presentation/SearchViewModel.kt

```
data class SearchUiState(
    val query: String = "",
    val repos: List<Repo> = emptyList(),
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

@OptIn(FlowPreview::class,
ExperimentalCoroutinesApi::class)
@HiltViewModel
class SearchViewModel @Inject constructor(
    private val searchRepos: SearchReposUseCase,
```

```
    private val refreshRepos: RefreshReposUseCase,
    private val savedState: SavedStateHandle
) : ViewModel() {

    private val query = savedState.getStateFlow("q",
"kotlin")
    private val isLoading = MutableStateFlow(false)
    private val error = MutableStateFlow<String?
>(null)

    private val validQuery = query
        .debounce(400)
        .filter { it.trim().length >= 2 }
        .distinctUntilChanged()

    private val repos = validQuery.flatMapLatest {
searchRepos(it) }

    val uiState: StateFlow<SearchUiState> =
        combine(query, repos.onStart {
emit(emptyList()) },
                isLoading, error) { q, list, loading,
err ->
            SearchUiState(q, list, loading, err)
        }.stateIn(viewModelScope,

SharingStarted.WhileSubscribed(5_000),
                  SearchUiState())

    init {
        viewModelScope.launch {
validQuery.collectLatest { refresh(it) } }
    }

    fun onQueryChange(q: String) { savedState["q"] =
q }

    fun retry() = viewModelScope.launch {
refresh(query.value) }

    private suspend fun refresh(q: String) {
        isLoading.value = true
        error.value = null
        refreshRepos(q).onFailure { e ->
            error.value = when (e) {
                ApiError.NoInternet -> "No internet.
```

```
Showing saved results."
                is ApiError.Http -> "Server error ($
{e.code})"
                else -> "Something went wrong"
            }
        }
        isLoading.value = false
    }
}
```

UI

presentation/SearchScreen.kt

```
@Composable
fun SearchRoute(vm: SearchViewModel =
hiltViewModel()) {
    val state by
vm.uiState.collectAsStateWithLifecycle()
    SearchScreen(state, vm::onQueryChange, vm::retry)
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SearchScreen(
    state: SearchUiState,
    onQueryChange: (String) -> Unit,
    onRetry: () -> Unit
) {
    Scaffold(topBar = { TopAppBar(title = { Text("🔍
Repo Explorer") }) }) { p ->
        Column(Modifier.padding(p).padding(horizontal
= 16.dp)) {
            OutlinedTextField(
                value = state.query,
                onValueChange = onQueryChange,
                label = { Text("Search
repositories") },
                singleLine = true,
                modifier = Modifier.fillMaxWidth())

            if (state.isLoading) {

LinearProgressIndicator(Modifier.fillMaxWidth()
                    .padding(vertical = 4.dp))
```

```
            }
            state.errorMessage?.let { msg ->
                Card(colors = CardDefaults.cardColors(

MaterialTheme.colorScheme.errorContainer),
                    modifier =
Modifier.fillMaxWidth().padding(vertical = 8.dp)) {
                    Row(Modifier.padding(12.dp),
                        verticalAlignment =
Alignment.CenterVertically) {
                        Text(msg, Modifier.weight(1f))
                        TextButton(onClick = onRetry)
{ Text("Retry") }
                    }
                }
            }
            LazyColumn(
                contentPadding =
PaddingValues(vertical = 8.dp),
                verticalArrangement =
Arrangement.spacedBy(8.dp)
            ) {
                items(state.repos, key = { it.id }) {
RepoCard(it) }
            }
        }
    }
}

@Composable
fun RepoCard(repo: Repo) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(12.dp)) {
            Row(verticalAlignment =
Alignment.CenterVertically) {
                AsyncImage(repo.avatarUrl, repo.owner,

Modifier.size(32.dp).clip(CircleShape))
                Spacer(Modifier.width(8.dp))
                Text("${repo.owner}/${repo.name}",
                    style =
MaterialTheme.typography.titleMedium,
                    maxLines = 1, overflow =
TextOverflow.Ellipsis)
            }
            repo.description?.let {
```

```
                Text(it, maxLines = 2, overflow =
TextOverflow.Ellipsis,
                     style =
MaterialTheme.typography.bodyMedium)
            }
            Row(Modifier.padding(top = 6.dp)) {
                Text("⭐ ${repo.stars}")
                Spacer(Modifier.weight(1f))
                repo.language?.let { Text("● $it") }
            }
        }
    }
}
```

✍️ **Try it yourself**

1) Add a **detail screen** with Navigation: tap a card → `DetailRoute(repo.id)` .

2) Add **favorites**: a ⭐ button that toggles `isFavorite` in Room.

3) Write a **ViewModel test** proving that the error message appears when the network fails.

⚡ Architecture Cheat Sheet

The modern Android stack on one page

Concept	Code
Layers	UI → ViewModel → UseCase → Repository → API / DAO
UI state	`data class XUiState(...)` + `StateFlow`
Update state	`_state.update { it.copy(...) }`
Collect in UI	`collectAsStateWithLifecycle()`
Hot from cold	`.stateIn(scope, WhileSubscribed(5_000), init)`
Combine	`combine(a, b) { x, y -> }`
Search	`debounce → distinctUntilChanged → flatMapLatest`
Parallel	`coroutineScope { async {}; async {} }`
Background	`withContext(Dispatchers.IO)`
Use case	`operator fun invoke()`
Retrofit	`@GET("path") suspend fun x(@Query q): Dto`
Room	`@Entity @Dao @Upsert @Transaction Flow<List>`
Hilt app	`@HiltAndroidApp` + `@AndroidEntryPoint`
Provide	`@Module @InstallIn @Provides @Singleton`
Bind interface	`@Binds abstract fun bind(impl): Interface`
ViewModel	`@HiltViewModel ... @Inject constructor`
Get VM	`hiltViewModel()`
Nav args	`savedStateHandle.toRoute<Route>()`
Test flows	`runTest { flow.test { awaitItem() } }`

ANSWERS

Exercise Solutions

Repo Explorer challenges



1) Detail screen

Detail.kt

```
@Serializable data object SearchDest
@Serializable data class DetailDest(val repoId: Long)

@Composable
fun AppNavHost() {
    val nav = rememberNavController()
    NavHost(nav, startDestination = SearchDest) {
        composable<SearchDest> {
            SearchRoute(onRepoClick = {
nav.navigate(DetailDest(it)) })
        }
        composable<DetailDest> {
            DetailRoute(onBack = nav::popBackStack)
        }
    }
}

@HiltViewModel
class DetailViewModel @Inject constructor(
    handle: SavedStateHandle,
    repository: RepoRepository
) : ViewModel() {
    private val id =
handle.toRoute<DetailDest>().repoId
    val repo = flow { emit(repository.getById(id)) }
        .stateIn(viewModelScope,
SharingStarted.WhileSubscribed(5_000), null)
}

@Composable
fun DetailRoute(onBack: () -> Unit,
                vm: DetailViewModel =
hiltViewModel()) {
    val repo by vm.repo.collectAsStateWithLifecycle()
    repo?.let {
```

```
        Column(Modifier.padding(16.dp)) {
            Text(it.name, style =
MaterialTheme.typography.headlineMedium)
            Text(it.description ?: "No description")
            Text("⭐ ${it.stars}   ● ${it.language ?:
"?"}")
            Button(onClick = onBack) { Text("Back") }
        }
    }
}
// In SearchScreen: add onRepoClick and Card(onClick =
{ onRepoClick(repo.id) })
```

2) Favorites

Favorites.kt

```
// RepoDao
@Query("UPDATE repos SET isFavorite = NOT isFavorite
WHERE id = :id")
suspend fun toggleFavorite(id: Long)

// RepoRepository (domain) + impl
suspend fun toggleFavorite(id: Long)
override suspend fun toggleFavorite(id: Long) =
dao.toggleFavorite(id)

// Use case
class ToggleFavoriteUseCase @Inject constructor(
    private val repository: RepoRepository) {
    suspend operator fun invoke(id: Long) =
repository.toggleFavorite(id)
}

// ViewModel
fun onFavoriteClick(id: Long) = viewModelScope.launch
{ toggleFavorite(id) }

// UI: add isFavorite to Repo + mapper, then in
RepoCard:
IconButton(onClick = { onFavoriteClick(repo.id) }) {
    Icon(if (repo.isFavorite) Icons.Filled.Star
         else Icons.Outlined.StarBorder, "Favorite")
}
```

> **📌 Remember**
> Because the UI observes Room, the star updates automatically — no manual state change needed. That's the power of a single source of truth.

3) ViewModel test

test/SearchViewModelTest.kt

```
class SearchViewModelTest {
    @get:Rule val mainRule =
MainDispatcherRule(StandardTestDispatcher())

    @Test
    fun `shows error when network fails`() = runTest {
        val repo = object : RepoRepository by
FakeRepoRepository() {
            override suspend fun refresh(query:
String) =

Result.failure<Unit>(ApiError.NoInternet)
        }
        val vm =
SearchViewModel(SearchReposUseCase(repo),

RefreshReposUseCase(repo),
                                    SavedStateHandle())
        vm.uiState.test {
            advanceUntilIdle()
            assertEquals("No internet. Showing saved
results.",

expectMostRecentItem().errorMessage)
            cancelAndIgnoreRemainingEvents()
        }
    }
}
```

```
▶ Test results
✔ shows error when network fails
```

🎉 You now know how professional Android apps are built!
Next steps: study Google's **Now in Android** sample, try Kotlin Multiplatform, add Firebase Crashlytics & analytics, and build your own portfolio app with this stack.